

01-406-093-203

การพัฒนาเว็บแอปพลิเคชันด้วยเฟรมเวิร์ค

โดย นายศุภมิตร จันทร์ทองศ์

- สาขาวิชาระบบสารสนเทศ คณะบริหารธุรกิจ

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

เช็คชื่อเข้าชั้นเรียน

- ให้นักศึกษาสแกน QR Code นี้เพื่อเข้าสู่ระบบเช็คชื่อเข้าเรียน
- ระบบจะทำการบันทึกประวัติการเข้าเรียนในวิชานี้อัตโนมัติ

สแกนเพื่อเช็คชื่อเข้าชั้นเรียน (สัปดาห์ที่ 14)



สแกน QR Code เพื่อเช็คชื่อเข้าเรียนของสัปดาห์นี้

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์โทวงศ์

สไลด์ที่ 2 / 32

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

แบบทดสอบก่อนเรียน (Pre-test)

1. คำสั่งใน BlogController ขั้ต่อไปนี้ส่งค่าความถี่สถานะเปิดใช้งาน โดยเรียงลำดับจากใหม่สุดมาแสดงที่หน้าแรก?

- A. Blog::orderByDesc('id')->where('status', 'true')->get();
- B. Blog::all()->where('status', 'false');
- C. Blog::orderBy('id')->get();
- D. Blog::where('status', 'draft')->all();

3. การกำหนด Route สำหรับรับรายละเอียดความถี่รับค่า ID ในไฟล์ routes/web.php ควรเป็นอย่างไร?

- A. Route::get('detail/{id}', [BlogController::class, 'detail']);
- B. Route::post('detail/{id}', [BlogController::class, 'detail']);
- C. Route::get('detail', [BlogController::class, 'detail']);
- D. Route::get('detail/{id}', [BlogController::class, 'detail']);

5. ไบรอนด์ Blade แบบใดที่ใช้สำหรับแสดงผลข้อมูลที่บันทึก HTML จาก Summernote โดยไม่แปลงให้เป็นข้อความดิบ (Unescaped)?

- A. <div>{!! \$blog->content !!}</div>
- B. <div>{{ \$blog->content }}</div>
- C. <div><%= \$blog->content %></div>
- D. <div>{{{ \$blog->content }}}</div>

7. การตัดทอนเนื้อหาความถี่ให้เหลือเพียง 100 ตัวอักษรและลบเท็ก HTML ใน Blade เขียนอย่างไร?

- A. {!! Str::limit(strip_tags(\$item->content), 100) !!}
- B. {{ Str::cut(strip_tags(\$item->content), 100) }}
- C. {{ Text::limit(\$item->content, 100) }}
- D. {{ String::short(strip_tags(\$item->content), 100) }}

9. ในการเปิดใช้งาน Summernote บนเท็ก <textarea name="content" id="content"> ต้องเรียกใช้งานผ่าน jQuery อย่างไร?

- A. \$('#content').summernote({});
- B. \$.summernote('#content');
- C. Summernote.init('#content');
- D. document.getElementById('content').summernote();

2. หากต้องการส่งข้อมูล \$blogs จาก BlogController ไปแสดงหน้าเว็บ welcome.blade.php ต้องใช้คำสั่งใด?

- A. return view('welcome', compact('blogs'));
- B. return redirect('welcome')->with('blogs');
- C. return view('welcome')->blogs(\$blogs);
- D. return response()->view('welcome', \$blogs);

4. ใน BlogController ฟังก์ชัน detail(\$id) คำสั่งใดใช้ค้นหาความถี่ตามรหัส ID จากฐานข้อมูล?

- A. \$blog = Blog::find(\$id);
- B. \$blog = Blog::search(\$id);
- C. \$blog = Blog::whereId(\$id)->all();
- D. \$blog = Blog::get(\$id);

6. ฟังก์ชันหรือ Helper ใดที่ใช้ตัดเท็ก HTML ออกจากข้อความก่อนนำไปจัดจำนวนตัวอักษรบนหน้าแรก?

- A. strip_tags(\$item->content)
- B. remove_html(\$item->content)
- C. clean_tags(\$item->content)
- D. clear_html(\$item->content)

8. Summernote เป็นเครื่องมือประเภทใดในการพัฒนาเว็บไซต์?

- A. Rich Text / WYSIWYG Editor สำหรับจัดรูปแบบข้อความในแบบฟอร์ม
- B. ฐานข้อมูลแบบ NoSQL
- C. สมมติฐาน State ฝั่ง Client
- D. ไทปส์สำหรับจัดการ Routing

10. การกำหนด callbacks ใน Summernote ให้มีฟังก์ชัน onPaste มีวัตถุประสงค์หลักเพื่ออะไร?

- A. ล้างฟอร์แมตหรือสไตล์แปลกปลอมเมื่อคัดลอกข้อความมาวาง (Paste as Plain Text)
- B. ป้องกันไม่ให้ผู้ใช้เพิ่มข้อความได้
- C. บันทึกข้อมูลลงฐานข้อมูลอัตโนมัติทันทีที่วางข้อความ
- D. แปลงข้อความที่วางให้กลายเป็นตัวพิมพ์ใหญ่ทั้งหมด

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

แสดงข้อมูลในหน้าแรก (Home Page Display)

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์ทองดี

สัปดาห์ 4 / 32

แสดงข้อมูลในหน้าแรก (Home Page Display)

- การนำข้อมูลสำคัญที่ดึงมาจากฐานข้อมูล (เช่น สินค้าแนะนำ หรือบล็อกล่าสุด) ไปจัดแสดงในส่วนหน้าของหน้าหลักของเว็บไซต์
- ใช้ตรรกะสั่งจัดเรียงข้อมูลตามเวลาล่าสุด (`latest()`) หรือดึงข้อมูลแบบสุ่มเพื่อนำเสนอผู้เยี่ยมชมหน้าแรก
- จัดวางรูปแบบโดยใช้การ์ดข้อมูลแบบตารางหรือกริด (Grid System) ในการแสดงหัวข้อย่อย
- ตัวอย่างการเขียนสืบทับข้อมูลบล็อกและพ่วงเปลี่ยนหน้าจอนำส่ง

code-example.php

```
// ดึงบล็อกล่าสุด 6 รายการไปจัดแสดงในหน้าแรก
public function home()
{
    $posts = DB::table('posts')->latest()->limit(6)->get();
    return view('home', compact('posts'));
}
```

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

การสร้าง Controller สำหรับจัดการบทความ (BlogController)

- เปิด Terminal แล้วรับคำสั่ง `php artisan make:controller BlogController`
- ระบบจะสร้างไฟล์ Controller ใหม่ที่ `app/Http/Controllers/BlogController.php`
- ใช้สำหรับจัดการตรรกะการแสดงผลบทความหน้าแรกและหน้ารายละเอียดบทความของระบบ

```
code-example.bash
```

```
php artisan make:controller BlogController
```

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์โทวงศ์

สัปดาห์ 6 / 32

การสร้างฟังก์ชัน index ใน BlogController (BlogController.php)

- เปิดไฟล์ app/Http/Controllers/BlogController.php
- สร้างฟังก์ชัน index() ภายในคลาส BlogController
- ส่งกลับมุมมองหน้าแรกด้วยคำสั่ง return view('welcome');

code-example.php

```
// app/Http/Controllers/BlogController.php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class BlogController extends Controller
{
    function index()
    {
        return view('welcome');
    }
}
```

การนำเข้า BlogController ใน routes/web.php

- เปิดไฟล์ routes/web.php
- เพิ่มคำสั่ง use App\Http\Controllers\BlogController; ไว้ที่ส่วนบนของไฟล์
- เพื่อเตรียมพร้อมสำหรับการเรียกใช้งาน BlogController ในเส้นทาง Routing

code-example.php

```
// routes/web.php  
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\AdminController;  
use Illuminate\Support\Facades\DB;  
use App\Http\Controllers\BlogController;
```

การกำหนดเส้นทางแรก (Route::get) ให้เรียกผ่าน BlogController

- เปิดไฟล์ routes/web.php
- แก้ไข Route หน้าแรก ("/) ให้เรียกผ่าน BlogController แรกๆ index
- เปลี่ยนจากฟังก์ชัน Closure แบบเดิมมาเป็น Route::get('/', [BlogController::class, 'index']);

code-example.php

```
// routes/web.php
Route::get('/', [BlogController::class, 'index']);
```

การดึงข้อมูลความล่าสุดผ่าน Model ใน BlogController

- เปิดไฟล์ app/Http/Controllers/BlogController.php
- นำเข้า Model ด้วยคำสั่ง use App\Models\Blog;
- ในฟังก์ชัน index() ดึงบทความที่มีสถานะเปิดใช้งาน โดยเรียงจากใหม่สุด ด้วยคำสั่ง Blog::orderByDesc('id')->where('status', 'true')->get();
- ส่งข้อมูล \$blogs ไปยังวิวหน้าแรกด้วย compact('blogs')

code-example.php

```
// app/Http/Controllers/BlogController.php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Blog;

class BlogController extends Controller
{
    public function index()
    {
        $blogs = Blog::orderByDesc('id')->where('status', 'true')->get();
        return view("welcome", compact('blogs'));
    }
}
```

การกำหนดหัวข้อเนื้อหาในหน้าแรก (welcome.blade.php)

- เปิดไฟล์ resources/views/welcome.blade.php
- กำหนดหัวข้อหน้าเว็บใน @section('title', 'หน้าแรกของเว็บไซต์')
- ภายใน @section('content') เพิ่มแท็ก <h2>บทความล่าสุด</h2> พร้อมเส้นกั้น <hr>

code-example.html

```
<!-- resources/views/welcome.blade.php -->
@extends('layouts.app')

@section('title', 'หน้าแรกของเว็บไซต์')

@section('content')
    <h2>บทความล่าสุด</h2>
    <hr>
@endsection
```

การวนลูปแสดงรายการบทความในหน้าแรก (welcome.blade.php)

- เปิดไฟล์ resources/views/welcome.blade.php
- ใช้คำสั่ง @foreach (\$blogs as \$item) เพื่อวนลูปแสดงบทความแต่ละรายการ
- แสดงข้อความด้วย <h2>{{{ \$item->title }}}</h2> และรายละเอียดเนื้อหาด้วย <p>{{{ \$item->detail }}}</p>
- เพิ่มปุ่ม/ลิงก์ อ่านเพิ่มเติม พร้อมเส้นคั่น <hr> และปิดท้ายลูปด้วย @endforeach

```
code-example.html<!-- resources/views/welcome.blade.php -->@extends('layouts.app')@section('title', 'หน้าแรกของเว็บไซต์')@section('content')<h2>บทความล่าสุด</h2><hr>@foreach ($blogs as $item)<h2>{{{ $item->title }}}</h2><p>{{{ $item->detail }}}</p><a href="#">อ่านเพิ่มเติม</a><hr>@endforeach@endsection
```

การตัดก่อนเนื้อหาบทความด้วย Str::limit ใน Blade (welcome.blade.php)

- เปิดไฟล์ resources/views/welcome.blade.php
- เปลี่ยนการแสดงผลเนื้อหาจาก <p>{{ \$item->detail }}</p> เป็น <p>{{ Str::limit(\$item->content, 30) }}</p>
- ฟังก์ชัน Str::limit ช่วยจำกัดความยาวของเนื้อหาบทความให้แสดงเพียง 30 ตัวอักษร เพื่อความกระชับและเป็นระเบียบบนหน้าแรก

code-example.html

```
<!-- resources/views/welcome.blade.php -->
@extends('layouts.app')

@section('title', 'หน้าแรกของเว็บไซต์')

@section('content')
    <h2>บทความล่าสุด</h2>
    <hr>
    @foreach ($blogs as $item)
        <h2>{{ $item->title }}</h2>
        <p>{{ Str::limit($item->content, 30) }}</p>
        <a href="#">อ่านเพิ่มเติม</a>
    </foreach>
    @endsection
```

แสดงเนื้อหาในหน้าเว็บ (Detail Page)

- หน้าต่างรับข้อมูลสำหรับเปิดดูบล็อก ข้อมูลสินค้า หรือหัวข้อข่าวสารแบบตัวเต็ม
- กำหนด Route แบบ Dynamic Parameter เพื่อใช้รองรับ ID หรือ Slug ของหัวข้อบล็อกนั้นๆ
- คำเตือน: หากข้อมูลจาก Rich Text Editor มีเกิน HTML ปกติ ต้องแสดงผลใน Blade ด้วยรูปแบบปีกกา `{!! $value !!}` แทนการใช้ `{{ $value }}` (เนื่องจากระบบปกติจะกรองและแปลงเป็น Text)
- การดึงข้อมูลและนำบทความเดี่ยวมาแสดงผล

```
// การเชื่อมต่อฐานข้อมูลใน Blade โดยเพิ่ม HTML
<div class="post-content">
    {!! $blog->content !!}
<!-- ข้อความจะแสดง /
</div>
```

សំណួរ 15 / 32

การกำหนด Route สำหรับหน้ารายละเอียดบทความ (routes/web.php)

- เปิดไฟล์ routes/web.php
- เพิ่ม Route สำหรับหน้ารายละเอียดบทความแบบรับค่าพารามิเตอร์ ID
- กำหนดด้วยคำสั่ง Route::get('detail/{id}', [BlogController::class, 'detail']);

code-example.php

```
// routes/web.php
// ผู้ใช้งาน
Route::get('/', [BlogController::class, 'index']);
Route::get('detail/{id}', [BlogController::class, 'detail']);
```

การเชื่อมโยงลิงก์ไปยังหน้ารายละเอียดบทความ (welcome.blade.php)

- เปิดไฟล์ resources/views/welcome.blade.php
- ปรับความยาวเนื้อหาตัวอย่างเป็น 100 ตัวอักษร ด้วย {{ Str::limit(\$item->content, 100) }}
- ผู้ลึกลงอ่านเพิ่มเติมไปยังหน้ารายละเอียดบทความด้วย id }}">อ่านเพิ่มเติม

code-example.html

```
<!-- resources/views/welcome.blade.php -->
@extends('layouts.app')

@section('title', 'หน้าแรกของเว็บไซต์')

@section('content')
    <h2>บทความล่าสุด</h2>
    <hr>
    @foreach ($blogs as $item)
        <h2>{{ $item->title }}</h2>
        <p>{{ Str::limit($item->content, 100) }}</p>
        <a href="/detail/{{ $item->id }}">อ่านเพิ่มเติม</a>
    <hr>
    @endforeach
@endsection
```

การสร้างฟังก์ชัน detail ใน BlogController (BlogController.php)

- เปิดไฟล์ app/Http/Controllers/BlogController.php
- สร้างฟังก์ชัน public function detail(\$id) เพื่อรับห้สบทความ
- ค้นหาบทความตาม ID ผ่านคำสั่ง \$blog = Blog::find(\$id);
- ส่งข้อมูล \$blog ไปแสดงผลที่หน้า view detail ด้วยคำสั่ง return view("detail", compact("blog"));

code-example.php

```
// app/Http/Controllers/BlogController.php
public function detail($id)
{
    $blog = Blog::find($id);
    return view("detail", compact("blog"));
}
```

การสร้างวิวแสดงรายละเอียดบทความ (detail.blade.php)

- สร้างไฟล์ใหม่ที่ resources/views/detail.blade.php
- สืบทอดโครงสร้างหลักด้วย @extends('layouts.app')
- กำหนดชื่อหน้าเว็บตามชื่อบทความด้วย @section('title') {{\$blog->title}} @endsection
- ภายใน @section('content') แสดงชื่อหัวข้อบทความด้วย <h1>{{\$blog->title}}</h1>

code-example.html

```
<!-- resources/views/detail.blade.php -->
@extends('layouts.app')

@section('title')
    {{$blog->title}}
@endsection

@section('content')
    <h1>{{$blog->title}}</h1>
@endsection
```

การแสดงผลละเอียดเนื้อหาบทความ (detail.blade.php)

- เปิดไฟล์ resources/views/detail.blade.php
- กำหนดหัวข้อหน้าเว็บใน @section('title') ด้วย {{blog->title}}
- ภายใน @section('content') แสดงหัวเรื่องบทความ <h1>{{blog->title}}</h1>
- เพิ่มเส้นคั่น <hr> และแสดงเนื้อหาบทความพร้อมคั่น HTML ด้วย <div>{!! \$blog->content !!}</div>

code-example.html

```
<!-- resources/views/detail.blade.php -->
@extends('layouts.app')

@section('title')
    {{blog->title}}
@endsection

@section('content')
    <h1>{{blog->title}}</h1>
    <hr>
    <div>{!! $blog->content !!}</div>
@endsection
```

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

กำหนดเนื้อหาด้วย Summernote

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์โทวงศ์

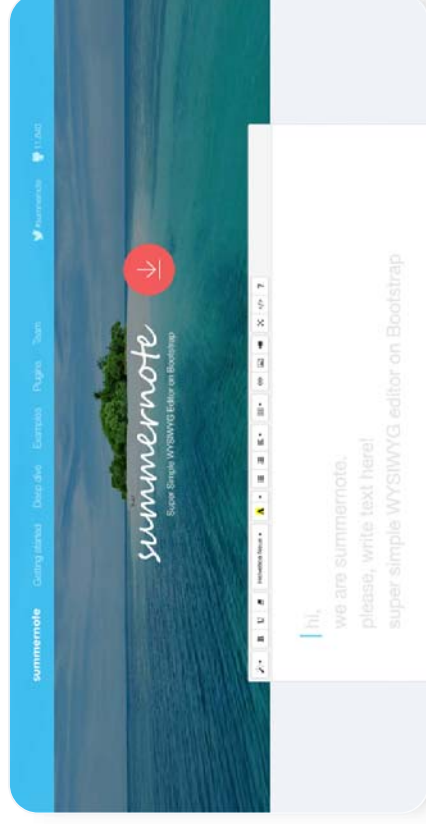
สัปดาห์ 21 / 32

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

เว็บไซต์หลักอย่างเป็นทางการของ Summernote (summernote.org)

- เข้าศึกษาคู่มือการติดตั้งและการใช้งานเพิ่มเติมได้ที่ summernote.org
- เครื่องมือ WYSIWYG Editor ใช้งานง่าย รองรับการจัดรูปแบบตัวอักษร, แทรกรูปภาพ, วิดีโอ และตาราง
- มีทั้งเวอร์ชันสำหรับ Bootstrap และ Summernote Lite แบบ Standalone ไม่ผูกติดกับ Framework



Summernote - Super Simple WYSIWYG Editor

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

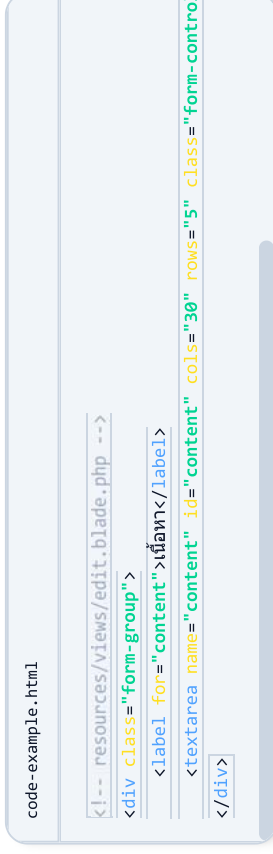
การติดตั้งและเปิดใช้งาน Summernote ใน app.blade.php

- เปิดไฟล์ resources/views/layouts/app.blade.php
- เพิ่ม jQuery CDN และ Summernote Lite (CSS & JS CDN) ก่อนปิดแท็ก </body>
- เขียนคำสั่ง JavaScript/jQuery เรียกใช้งาน \$('#content').summernote({ ... })
- กำหนด placeholder: 'เขียนเนื้อหาบทความที่นี่...', tabsize: 2 และ height: 250

```
code-example.html
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<!-- Summernote Lite CSS & JS CDN -->
<link href="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.js"></script>
$(document).ready(function() {
    $('#content').summernote({
        placeholder: 'เขียนเนื้อหาบทความที่นี่...',
        tabsize: 2,
        height: 250
    });
});
</script>
```

การผูก Summernote เข้ากับแบบฟอร์มแก้ไขบทความ (edit.blade.php)

- เปิดไฟล์ resources/views/edit.blade.php
- ที่แท็ก <textarea> สำหรับกรอกเนื้อหาบทความ ให้เพิ่มแอตทริบิวต์ id="content"
- เพื่อให้ตรงกับตัวเลือก \$('#content') ที่กำหนดไว้ในสคริปต์ Summernote ของ layouts/app.blade.php



การผูก Summernote เข้ากับแบบฟอร์มเพิ่มบทความ (form.blade.php)

- เปิดไฟล์ resources/views/form.blade.php
- ที่แท็ก <textarea> สำหรับกรอกเนื้อหาบทความ ให้เพิ่มแอตทริบิวต์ id="content"
- เพื่อให้กล่องข้อความแปลงเป็น Summernote Rich Text Editor ตามที่กำหนดใน layouts/app.blade.php

code-example.html

```
<!-- resources/views/form.blade.php -->
<div class="form-group">
  <label for="content">เนื้อหา</label>
  <textarea name="content" id="content" cols="30" rows="5" class="form-control">
</div>
```

การแสดงผลเนื้อหา HTML จาก Summernote ในหน้ารายละเอียดบทความ (detail.blade.php)

- เปิดไฟล์ resources/views/detail.blade.php
- เนื่องจาก Summernote บันทึกข้อมูลในรูปแบบแท็ก HTML (เช่น , <i>, <p>,)
- ให้แสดงผลด้วยไวยากรณ์ Blade Unescaped <div>{!! \$blog->content !!}</div>
- เพื่อให้เบราว์เซอร์เรนเดอร์โครงสร้างและรูปแบบข้อความตามที่จัดไว้ใน Summernote ได้อย่างถูกต้องสมบูรณ์

code-example.html

```
<!-- resources/views/detail.blade.php -->
@extends('layouts.app')

@section('title')
    {{$blog->title}}
@endsection

@section('content')
    <h1>{!! $blog->title !!}</h1>
    <hr>
    <div>{!! $blog->content !!}</div>
@endsection
```

การตัดแท็ก HTML ด้วย strip_tags ก่อนย่อเนื้อหาในหน้าแรก (welcome.blade.php)

- เปิดไฟล์ resources/views/welcome.blade.php
- ใช้ฟังก์ชัน strip_tags() ครอบตัวแปร \$item->content ก่อนส่งเข้า Str::limit
- เขียนเป็น {{ Str::limit(strip_tags(\$item->content), 100) }}
- ช่วยกำจัดแท็ก HTML ของ Summernote ออกทั้งหมด เหลือเพียงข้อความตัวอักษรล้วนสำหรับแสดงตัวอย่างในหน้าแรกได้อย่างสวยงามและเป็นระเบียบ

code-example.html

```
<!-- resources/views/welcome.blade.php -->
@extends('layouts.app')

@section('title', 'หน้าแรกของเว็บไซต์')

@section('content')
    <h2>บทความล่าสุด</h2>
    <hr>
    @foreach ($blogs as $item)
        <h2>{{ $item->title }}</h2>
        <p>{{ Str::limit(strip_tags($item->content), 100) }}</p>
        <a href="/detail/{{ $item->id }}">อ่านเพิ่มเติม</a>
    <hr>
    @endforeach
@endsection
```

การตั้งค่า onPaste ใน Summernote เพื่อล้าง ฟอร์แมตข้อความ (layouts/app.blade.php)

- เปิดไฟล์ resources/views/layouts/app.blade.php
- เพิ่ม callbacks: { onPaste: function(e) { ... } } ภายในการตั้งค่า Summernote
- ดึงข้อความล้วนด้วย clipboardData.getText() และสั่ง insertText
- ช่วยป้องกันไม่ให้ใส่โค้ดสคริปต์ได้แต่แลกเปลี่ยนจากเว็บอื่นขึ้นมาเมื่อผู้ใช้งานคัดลอกและวางข้อความลงในช่องเนื้อหา

code-example.js

```
// resources/views/layouts/app.blade.php
<script>
$(document).ready(function() {
    $('#content').summernote({
        placeholder: 'เขียนเนื้อหาความที่นี่...',
        tabsize: 2,
        height: 250,
        callbacks: {
            onPaste: function (e) {
                var bufferText = ((e.originalEvent || e).clipboardData || window.clipboardData).getData('Text');
                e.preventDefault();
                document.execCommand('insertText', false, bufferText);
            }
        }
    });
});
```

สัปดาห์ที่ 14 · การแสดงผลเนื้อหาบนหน้าเว็บและการใช้ SUMMERNOTE

01-406-093-203 LARAVEL COURSE

แบบทดสอบหลังเรียน (Post-test)

1. คำสั่งใน BlogController ข้างล่างจะดึงบทความที่มีสถานะเปิดใช้งาน โดยเรียงลำดับจากใหม่สุดมาแสดงที่หน้าแรก?

- A. Blog::orderByDesc('id')->where('status', 'true')->get();
- B. Blog::all()->where('status', 'false');
- C. Blog::orderBy('id')->get();
- D. Blog::where('status', 'draft')->all();

3. การกำหนด Route สำหรับรับรายละเอียดความถี่รับค่า ID ในไฟล์ routes/web.php ควรเป็นอย่างไร?

- A. Route::get('detail/{id}', [BlogController::class, 'detail']);
- B. Route::post('detail/{id}', [BlogController::class, 'detail']);
- C. Route::get('detail', [BlogController::class, 'detail']);
- D. Route::get('detail/{id}', [BlogController::class, 'detail']);

5. ไบรอนด์ Blade แบบใดที่ใช้สำหรับแสดงข้อมูลที่เก็บจาก HTML จาก Summernote โดยไม่แปลงให้เป็นข้อความดิบ (Unescaped)?

- A. <div>{!! \$blog->content !!}</div>
- B. <div>{{ \$blog->content }}</div>
- C. <div><%= \$blog->content %></div>
- D. <div>{{{ \$blog->content }}}</div>

7. การตัดทอนเนื้อหาบทความให้เหลือเพียง 100 ตัวอักษรและลบแท็ก HTML ใน Blade เขียนอย่างไร?

- A. {!! Str::limit(strip_tags(\$item->content), 100) !!}
- B. {{ Str::cut(strip_tags(\$item->content), 100) }}
- C. {{ Text::limit(\$item->content, 100) }}
- D. {{ String::short(strip_tags(\$item->content), 100) }}

9. ในการเปิดใช้งาน Summernote บนแท็ก <textarea name="content" id="content"> ต้องเรียกใช้งานผ่าน jQuery อย่างไร?

- A. \$('#content').summernote({ ... });
- B. \$.summernote('#content');
- C. Summernote.init('#content');
- D. document.getElementById('content').summernote();

2. หากต้องการส่งข้อมูล \$blogs จาก BlogController ไปแสดงหน้าเว็บ welcome.blade.php ต้องใช้คำสั่งใด?

- A. return view('welcome', compact('blogs'));
- B. return redirect('welcome')->with('blogs');
- C. return view('welcome')->blogs(\$blogs);
- D. return response()->view('welcome', \$blogs);

4. ใน BlogController พังค์ชัน detail(\$id) คำสั่งใดที่ใช้ค้นหาความถี่ตามรหัส ID จากฐานข้อมูล?

- A. \$blog = Blog::find(\$id);
- B. \$blog = Blog::search(\$id);
- C. \$blog = Blog::whereId(\$id)->all();
- D. \$blog = Blog::get(\$id);

6. พังค์ชันหรือ Helper ใดที่ใช้ตัดแท็ก HTML ออกจากข้อความก่อนนำไปจัดจําแนกด้วยอักขระบนหน้าแรก?

- A. strip_tags(\$item->content)
- B. remove_html(\$item->content)
- C. clean_tags(\$item->content)
- D. clear_html(\$item->content)

8. Summernote เป็นเครื่องมือประเภทใดในการพัฒนาเว็บไซต์?

- A. Rich Text / WYSIWYG Editor สำหรับจัดรูปแบบข้อความในแบบฟอร์ม
- B. ฐานข้อมูลแบบ NoSQL
- C. ระบบจัดการ State สำหรับ Client
- D. ไทปากรสำหรับจัดการ Routing

10. การกำหนด callbacks ใน Summernote ให้มีฟังก์ชัน onPaste มีวัตถุประสงค์หลักเพื่ออะไร?

- A. ล้างฟอร์แมตหรือสไตล์แปลกปลอมเมื่อคัดลอกข้อความมาวาง (Paste as Plain Text)
- B. ป้องกันไม่ให้ผู้ใช้เพิ่มข้อความได้
- C. บันทึกข้อมูลจากฐานข้อมูลอัตโนมัติทันทีที่วางข้อความ
- D. แปลงข้อความที่วางให้กลายเป็นตัวพิมพ์ใหญ่ทั้งหมด

คำถามท้ายบทเรียน

ไวยากรณ์ปีกกาของ Blade แบบใดที่ใช้สำหรับการแสดงข้อความดิบที่มีแท็ก HTML ปนอยู่ โดยไม่อ่านแปลงค่าอักขร (Unescaped)?

A. {{ \$content }}

B. {!! \$content !!}

C. <% \$content %>

D. {{{ \$content }}}

คำอธิบาย: ไวยากรณ์ {!! \$content !!} หรือ {!! \$blog->content !!} ใช้เพื่อเรนเดอร์ข้อความดิบที่มีสัญลักษณ์ HTML ปนอยู่ให้เป็นภาพผลลัพธ์จริงบนหน้าจอเบราว์เซอร์

Workshop 14: ทำหน้าข่าวสาร และติดตั้งตัวตกแต่งเนื้อหาด้วย Summernote

- 1 ดึงข้อมูลจากตารางข่าวสารออกมาเรียงลำดับแสดงที่หน้าแรก (Home Page)
- 2 ทำหน้าต่างบทความเดี่ยว แสดงข้อมูลแบบไม่กรองป้ายแก้ไขความสวยงาม {!! \$blog->content !!} >content !!}
- 3 เพิ่มสคริปต์เรียกใช้งาน Summernote Lite และ jQuery ผ่านระบบ CDN ในไฟล์ layouts/app.blade.php
- 4 ตั้งค่ากล่องรับเนื้อหาแบบ textarea (#content) ให้กลายเป็น Rich Text Editor เพื่อจัดการฟอร์แมตข้อมูล

command_prompt.sh

```
<!-- เปิดใช้การแสดงผลแบบ HTML แฝงใน Blade -->
<div>{!! $blog->content !!}</div>
```

ใบงานที่ 14: ระบบเขียนบล็อกและแสดงเนื้อหาเต็มรูปแบบพร้อมตัวตกแต่ง Summernote

กำหนดบันทึกบทความที่ผู้เขียนสามารถใช้ Summernote จัดหัวข้อ ทำตัวหนา หรือขีดเส้นใต้ได้ และสามารถนำบทความนั้นไปแสดงผลที่หน้าแรกได้อย่างสมบูรณ์

เกณฑ์การตรวจประเมินผล:

การเชื่อมต่อและแสดงบทความบนหน้าหลักได้อย่างถูกต้อง

40%

การติดตั้งและเรียกใช้งานหน้าต่าง Summernote ลงในระบบได้สมบูรณ์

40%

การแสดงผลในหน้าวิวเต็มรูปแบบอย่างถูกต้องปลอดภัย

20%

